

Volumes =====

Volumes are unity catalog objects that enable governance over non-tabular datasets.

while tables govern tabular data, volumes govern non-tabular data of any format, including structured, semi structured or unstructured.

Databricks recommends using volumes to govern all non tabular data.

2 types of volumes

- Managed
- external

to create a external volume you should have a external location configured

azure cloud

externaldata -> ext_volume

abfss://externaldata@ttmystorageaccount001.dfs.core.windows.net/ext_volume

Differences between managed and external volume

- storage location

- data lifecycle

- access control

for managed volumes all access goes through UC

for external UC governs access, but external tools can use direct URI's

- typical usecase

managed volumes - simplest option for databricks only workloads

external volumes - mixed databricks and external system access.

what is the purpose of external volumes when we have external locations on cloud paths.

external locations allow us to access data stored outside of databricks,

volumes provide a way to manage access to specific subfolders within those external locations.

external volumes can help us manage granular/subfolder level access and not the main folder access.

DBFS mountpoints

managing access to external cloud object storage transitioned from DBFS mount points to UC volumes.

Better data governance and how they integrate with broader databricks ecosystem.

there is a one month free trial for your azure account (\$200)

free edition

azure databricks

I am assuming you would need more than 1 month, 4 months to practise.

azure give 30 days free trial

databricks workspace free trial - 14 days free DBU's

azure subscription is over

databricks free trial - 14 days (classic compute)

microsoft azure will charge you for the hardware

DBU's will be free..

azure subscription is over

databricks free trial - 14 days (serverless compute)

serverless the hardware also comes from databricks

DBU's will be free..

Deploy Azure Databricks workspace with Secure Cluster Connectivity (No Public IP) - by default it is Yes

create a service - NAT Gateway (\$1.5 per day)

It ensures that databricks clusters, SQL warehouse and other compute do not have public IP address.

Instead, all communication happens through private network channels improving security and compliance.

create a budget - \$5 it resets monthly.

alert based on this budget

use free edition whenever possible

create a free trial for databricks workspace (14 days)

when creating this workspace then make sure the

Deploy Azure Databricks workspace with Secure Cluster Connectivity (No Public IP) - by default it is Yes, make it to No

use serverless compute.

make sure do not fill up your storage, keep deleting bigger files.

set the budget and alerts based on that.

=====

Lakeflow - Connect / Declarative Pipelines / Jobs

Data Processing Engine (Apache Spark)

Unified Governance (Unity Catalog)

Optimized Storage (Delta Lake)

Delta Lake

- ACID properties
- DML
- Time Travel
- Schema Evolution and Enforcement

Lakeflow - Connect / Declarative Pipelines / Jobs

Lakeflow provides a unified platform for

- data ingestion (lakeflow connect)
- transformation (lakeflow declarative pipeline) DLT
- orchestration (lakeflow jobs) workflows

data ingestion (lakeflow connect)

=====

earlier the organizations have to use a lot of third party tools and inhouse solutions for data ingestions.

cloud storages - adls gen2 , amazon s3, databricks volumes

databases

SaaS applications

with lakeflow connect we can have efficient ingestion pipelines all within databricks

Cloud storage

Delta Lake

SaaS application

you ingest to delta lake

and once data is in delta lake you do ETL

Medallion architecture

Bronze -> Silver -> Gold -> (Consumers)

Lakeflow connect

different types of connectors

- upload files from local storage to volume
- Standard connectors (ingest from cloud storage into your delta lake)
- managed connectors (ingest from SaaS applications / Databases)

Standard connectors

=====

different kind of ingestion modes

- batch ingestion (all data is re-ingested each time)
- incremental batch (all new data is ingested, previously loaded records are skipped automatically)
- incremental streaming - continuously load data rows or batches of data rows as it is generated so that you can query as it arrives in near real time.

in next session we will see how to do data ingestion from cloud storage.

Data Ingestion from cloud storage

- CTAS
- Copy Into
- Auto Loader

Lakeflow connect standard connectors

convert raw file into delta tables

- CTAS

Create table as select *

All the data is re ingested each time.

Mode

- permissive

the mode defines how spark should behave when it encounters bad or malformed records.

permissive - keeps the job running, and put the bad rows into `_rescued_data` column.

dropmalformed - silently drops bad records.

failfast - aborts on the first bad row.

=====

CTAS

- full load
- do not support incremental batch

copy into
=====

loads data from cloud file location into delta lake.

This is a retrievable and idempotent operation.

exactly once processing.

incremental batch - files in the source location that have already been loaded are skipped.

- incremental batch
- by creating an empty table without schema
- we could have created table with schema as well
- format_options() control how the source files are parsed and interpreted
- copy_options() controls the behaviour of copy operations itself. such as schema evolution (mergeSchema)

copy into is best suited when we are dealing with upto thousands of files.

Lakeflow connect standard connectors

- CTAS
- Copy into (incremental batch)

scenarios which copy into command cannot handle.

- what if a new file is coming in every 5 mins, every 2 mins or even more frequently
- if millions of files are coming each hour, or you have to backfill billions of files.

that's where autoloader is needed. it's a more advanced technique than copy into to incrementally and efficiently process new data files as they arrive in cloud storage.

- autoloader can handle both incremental batch and streaming.

- you can use auto loader to process billions of files to migrate or backfill a table.
auto loader scales to support near real time ingestion of millions of files per hour.

autoloader can load data files from any of the cloud source

- amazon s3
- adls gen2
- GCS

volume path

/Volumes/deltalake_catalog/default/raw

folder - ordershistory

/Volumes/deltalake_catalog/default/raw/ordershistory/

writeStream is a unified API for both batch and streaming

.trigger(processingTime = 10 seconds) //streaming

.trigger(availableNow=True) //batch

how does autoloader track the ingestion progress?

as files are discovered, their metadata is persisted in a scalable key value store (RocksDB) in the checkpoint location.

This key value store ensures that data is processed exactly once.

in case of failures, autoloader can resume from where it left off by information stored in the checkpoint location.

This is what helps us to get exactly once semantics.

=====

I will quickly reiterate the code

Batch style

Streaming mode

=====

1. how schema inference works internally
2. what if you want to manually define the schema
3. Schema hints
4. add new metadata columns

1000 files or 50 GB of data whatever comes first

1000 files each of 1 mb = 1 GB

10 files of 5 gb each = 50 GB

`cloudFiles.schemaInference.sampleSize.numBytes`

`cloudFiles.schemaInference.sampleSize.numFiles`

Schema Evolution Modes

- none
- fail on new columns
- rescue

- add new columns (default)

none
=====

Does not evolve the schema, new columns are ignored, and data is not rescued unless the rescuedDataColumn option is set. Stream does not fail due to schema changes.

fail on new columns
=====

stream fails, stream does not restart unless the provided schema is updated, or the offending data file is removed.

rescue
=====

schema is never evolved and stream does not fail due to schema changes. all new columns are recorded in the rescued data column.

add new columns (default)
=====

stream fails, new columns are added to the schema. existing columns do not evolve data types.

Note: addNewColumns mode is the default mode when a schema is not provided.

but none is the default mode when you provide a schema.